A decorative graphic on the left side of the slide, consisting of a black crosshair with a blue square in the top-left quadrant, a red square in the bottom-left quadrant, and a yellow square in the bottom-right quadrant.

Computer Programming II (CSE2035)

Week 7 Lab Session:

Object Oriented Programming Part 3

Dept. of Computer Science and Engineering
Sogang University

Exercise 0: Lecture Management

- Now let's write classes that interact with Lecture in which you implemented from the last lab session
- **First, copy your w6_exercise_0/lecture.cpp file (previous lab assignment) into w7_exercise_0/**
- UnivMem class is defined in univmem.hpp (**do not fix this file**) and this class will be inherited by Student and Professor classes

```
class UnivMem {
private:
    int id;
    char name[MAX_NAME_LEN + 1];
public:
    UnivMem(int _id, const char* _name) { ... }
    int GetID() { ... }
    const char* GetName() { ... }
    void Print() { ... }
```

Exercise 0: Lecture Management

- Student class is partially defined in `student.hpp` (**do not fix this file**) by inheriting `UnivMem`
 - Properties and constructor are given
 - 'lecture_cnt' stores how many lectures this student is taking
 - 'Lecture' stores object pointers of the lectures that this student is taking
 - A student can take up to `MAX_LECTURE` lectures

```
class Student: public UnivMem {
private:
    int lecture_cnt;
    Lecture* lectures[MAX_LECTURE];
public:
    Student(int _id, const char* _name) { ... }
    ...
}
```

Exercise 0: Lecture Management

- Implement the following functions in `student.cpp`

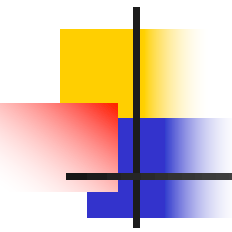
```
// Print the grades of this student. For each of the lecture,
// print its lecture code and the student's grade in the
// following format:
// "<lecture code> : <grade string>\n"
// You can use GradeToString() function provided in lecture.hpp
void PrintGrade();

// If this student can take more lectures, try to add this
// student to 'lec'. Return false if this registration was
// unsuccessful. If the registration was successful, update
// 'lectures' and 'lecture_cnt' properties to reflect the
// registration, and then return true.
bool Register(Lecture *lec);
```

Exercise 0: Lecture Management

- Next, we define Professor class as well
- Professor class is partially defined in professor.hpp file (**do not change this file**) by inheriting UnivMem
 - Properties and constructor are given
 - A professor always has one lecture to teach

```
class Professor: public UnivMem {  
private:  
    Lecture *lecture;  
public:  
    Professor(int _id, const char* _name, Lecture * _lecture)  
    { ... }  
    ...  
}
```



Exercise 0: Lecture Management

- Implement the following function in `professor.cpp`

```
// Update the student's grade in 'lecture'. Set the grade of  
// the student with 'student_id' into 'student_grade'. Return  
// false if such a student does not exist in 'lecture'.  
bool GradeStudent(int student_id, Grade student_grade);
```

Exercise 0: Lecture Management

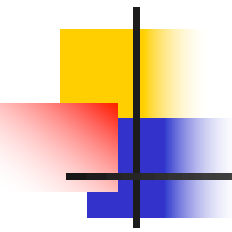
- In `main.cpp`, you can find a test function
 - You can freely change this function to test your class implementation in various ways

```
void test() {
    Lecture lec1("CSE2015");
    Lecture lec2("CSE2035");
    Professor p(102233, "Jaeseung", &lec1);
    Student s1(20220001, "Alice");
    Student s2(20220002, "Brian");

    s1.Register(&lec1);
    s1.Register(&lec2);
    p.GradeStudent(20220001, B);
    s1.Print();
    s1.PrintGrade();
    ...
}
```

(Expected Output)

```
Alice(20220001)
CSE2015 : B
CSE2035 : NA
```



Lab Assignment Submission Instructions

- **Create “WeekX_Lab_yourID_name” folder**
 - E.g., Week7_Lab_20221234_문의현
- **Put your completed works (e.g., codes/files/programs) to the folder**
- **Compress (Zip) your folder**
 - E.g., Week7_Lab_20221234_문의현.zip
- **Submit/Upload your Zip file to the Cyber Campus in the “Assignments” section**
- **Due dates: All the lab assignments are due by Wednesday at 11:59pm, right after prior week’s lab session on Thursday**
 - Late submissions are not accepted